1. 3D分拣场景进入与介绍

1.1 环境搭建-安装证书

证书是为了让元宇宙平台的3D场景能够找到咱们的电脑的服务中。

1. 打开**华清远见人工智能虚拟仿真本地服务管理平台**，点击右上角的**全局设置**按钮。

2. 在弹出来的下拉框中，点击**安装证书**按钮。

华清远见元宇宙实验中心 yyzlab.com.cn

3. 在弹出的提示框中，点击**安装**按钮。

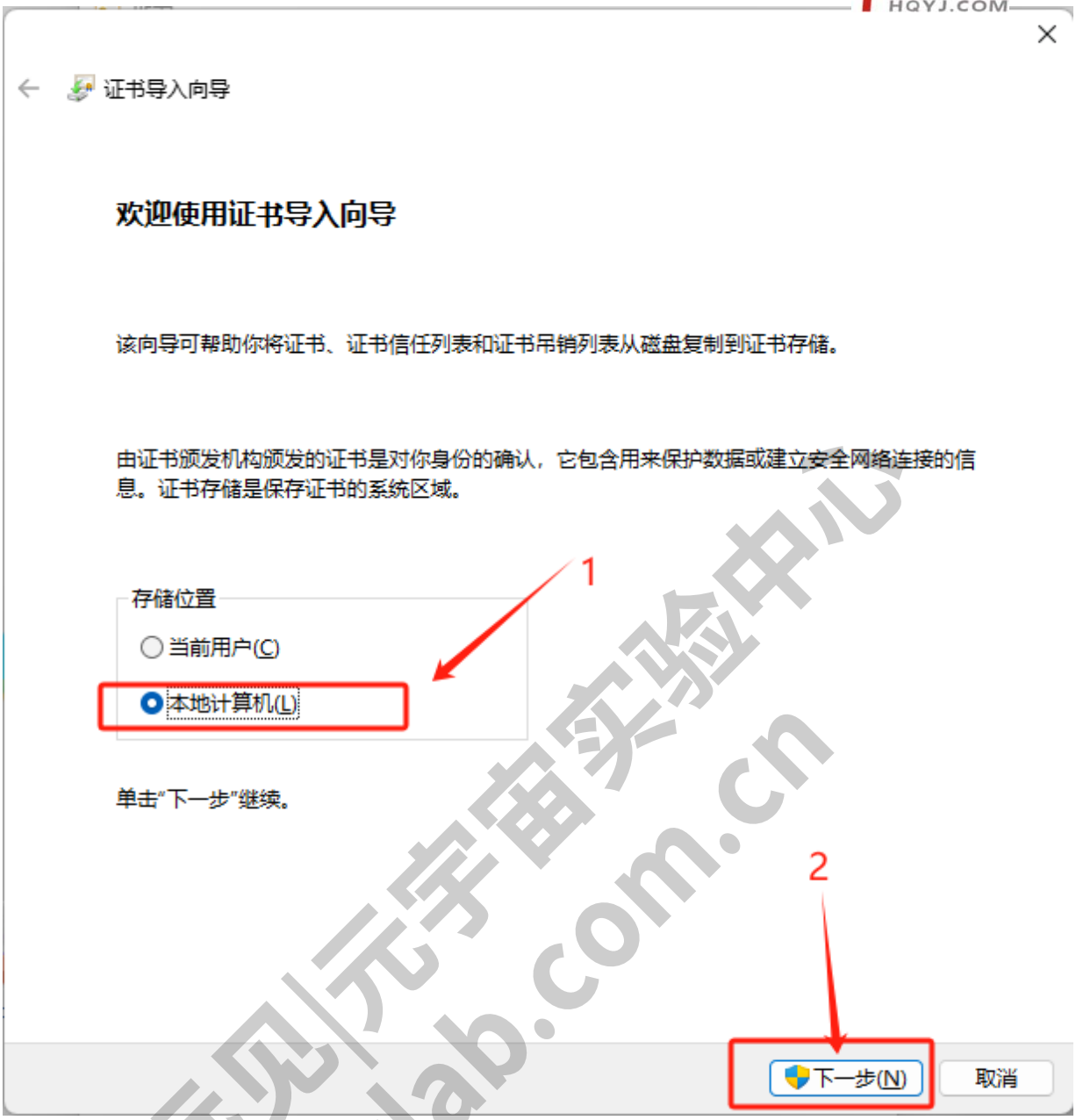
华清远见元宇宙实验中心
yyzlab.com.cn

4. 在弹出来的窗口中，点击**安装证书**按钮。

华清远见|元宇宙实验中心
yyzlab.com.cn

5. 在弹出来的窗口中，点击**本地计算机**按钮，然后点击**下一步**。

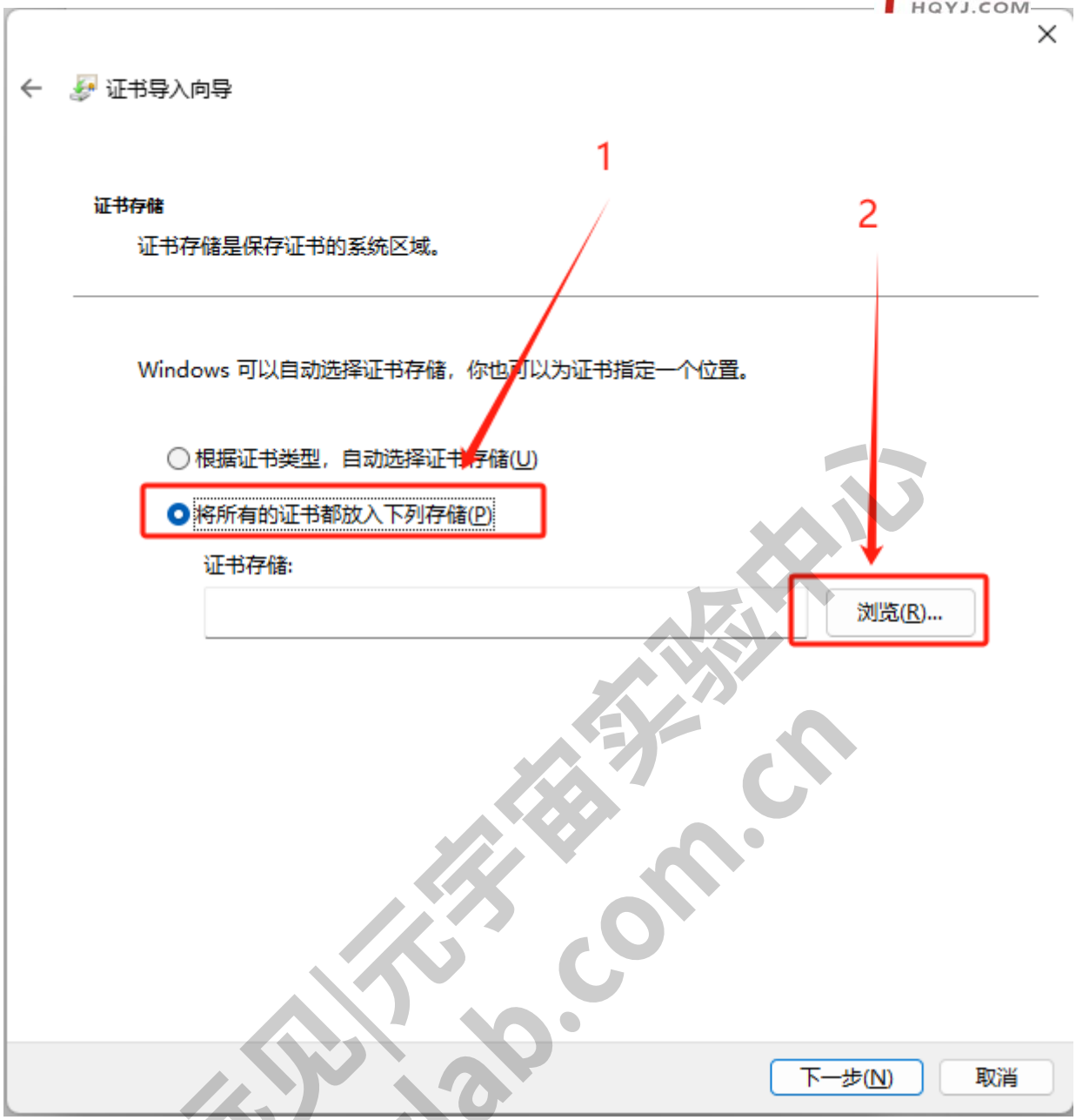
华清远见 HQYJ.COM
华清远见|元宇宙实验中心 yyzlab.com.cn



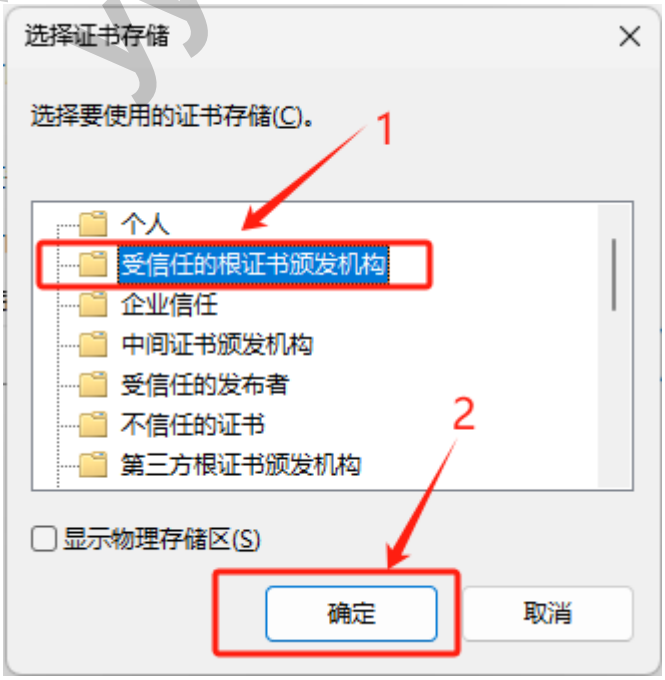
5. 在弹出来的窗口中，选择**将所有的证书都放入下列存储**选项，然后点击**浏览**按钮。

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见
HQYJ.COM



6. 在弹出来的窗口中，选择**受信任的根证书颁发机构**选项，然后点击**确定**按钮。



华清远见|元宇宙实验中心
yyzlab.com.cn

8. 接着点击**下一步**按钮。

9. 点击**完成**按钮，等待他自动安装证书。

华清远见元宇宙实验中心 yyzlab.com.cn

10. 弹出**导入成功**窗口后，点击确定，即表示证书安装成功。

华清远见|元宇宙实验中心
yyzlab.com.cn

1.2 进入3D场景

1.2.1 打开元宇宙实验中心

打开网页，在网页中输入网址：www.yyzlab.com.cn 进入元宇宙实验中心。

华清远见元宇宙实验中心 yyzlab.com.cn

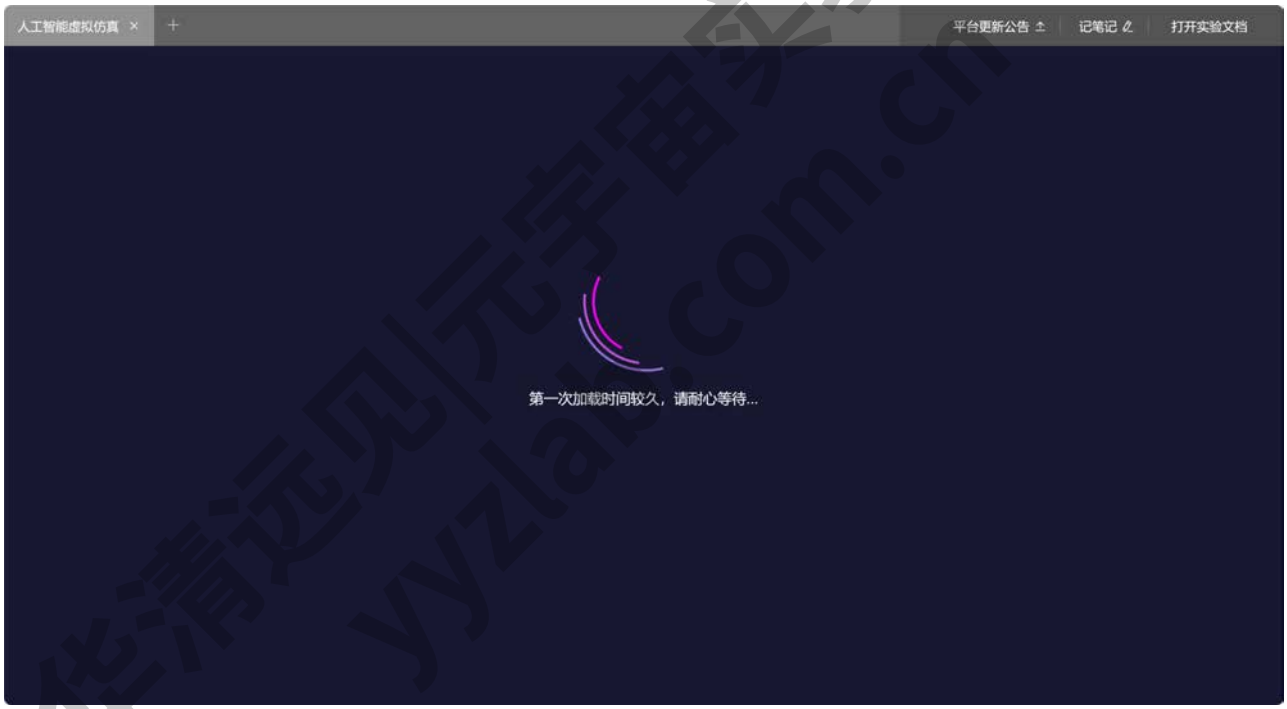
1.2.2 进入人工智能虚拟仿真

1. 点击元宇宙实验中心首页的**在线实验平台**。

2. 在跳转到的页面中，点击**人工智能虚拟仿真**，进入**人工智能虚拟仿真系统。**

华清远见|元宇宙实验中心 yyzlab.com.cn

3. 点击**马上开启实验**按钮，进行加载资源。



4. 点击**马上进入**按钮，进入系统中。

华清远见|元宇宙实验中心 yyzlab.com.cn

1.2.3 进入3D分拣场景

1. 进入到**人工智能虚拟仿真系统**中，点击左侧的**综合项目**按钮。

2. 在跳转的页面中，点击**智能分拣系统。**

华清远见实验平台 yyzlab.com.cn

3. 点击**进入3D场景**按钮，进入3D场景。

1.3 3D场景的介绍

整个3D场景共分为3个模式，分别为**采集模式**，**分拣模式**，**考试模式**，整个场景使用**MQTT**的通信方式去进行通信。

采集模式：主要用于数据的采集，需要通过代码去接收采集模式所传递出来的照片。

分拣模式：需要通过代码控制，主要用于算法的整体部署及场景的逻辑控制。

考试模式：需要通过代码控制，主要用于考核成绩，检验算法的准确率及逻辑的控制能力。

华清远见宇宙实验中心 yyzlab.com.cn

1.3.1 采集模式

点击**采集模式**按钮，进入**采集模式**。

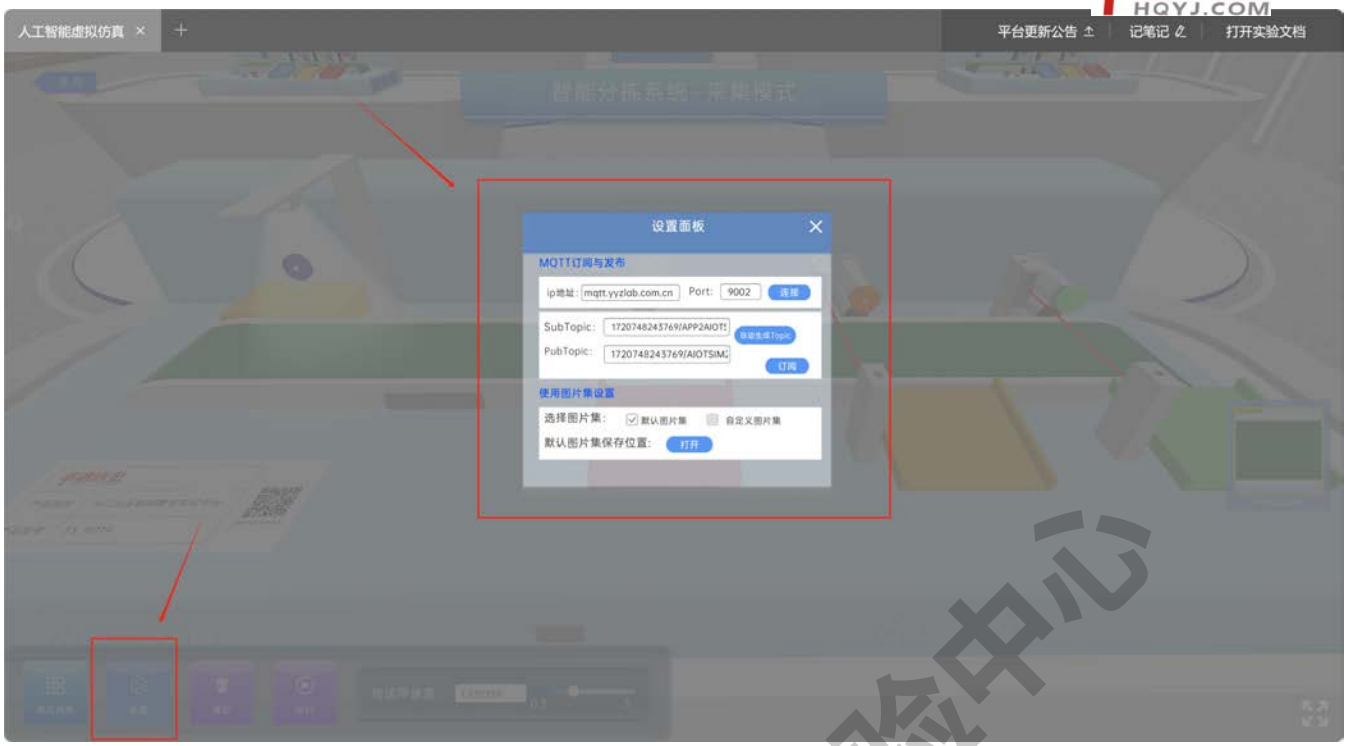
在该模式下，可以通过键盘上的**W A S D**键进行前后左右的移动和通过**按住鼠标右键进行拖动**来调整视角，通过这两个功能，可以帮助我们将视角移到**分拣台**上。

华清远见元宇宙实验中心 yyzlab.com.cn

1. 在当前页面中，从左下角开始，第一个按钮为**模式选择**，点击该按钮，可以让我们在场景内自由切换模式。

2. 第二个按钮为**设置**按钮，点击该按钮，会弹出**设置面板**，在该面板中，需要我们去设置要连接的MQTT的**IP地址**、**端口号**以及**SubTopic和PubTopic**。

华清远见元宇宙实验中心 yyzlab.com.cn

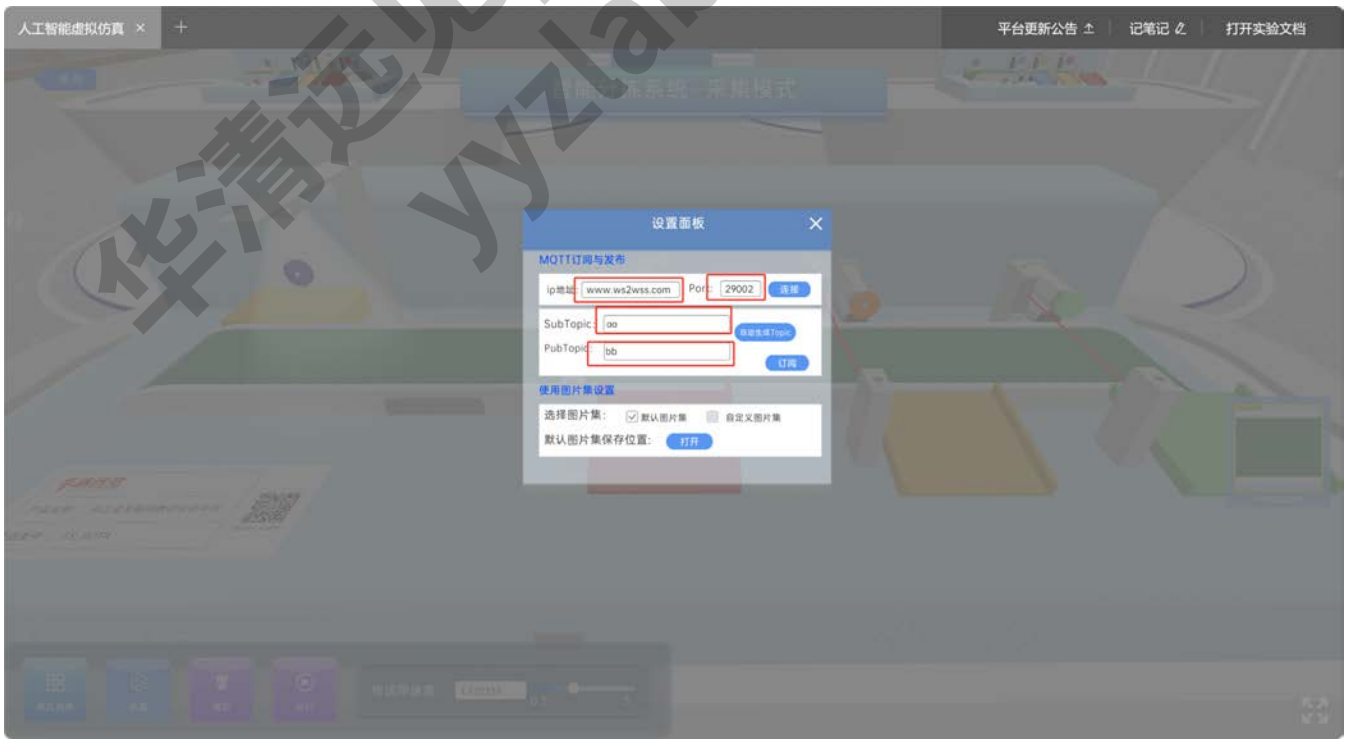


在这里，默认的**IP地址**、**端口号**以及**SubTopic和PubTopic**是不能用的，我们需要进行修改：

IP地址修改为：www.ws2wss.com或**127.0.0.1**

端口号修改为：**29002**

SubTopic和PubTopic：这个随意修改，但进来改成比较简单的，比如aa和bb，如下图所示。



在**华清远见人工智能虚拟仿真本地服务管理平台**启动的状态下，可以点击**连接**和**订阅**按钮，进行连接服务和订阅话题，会出现下图所示现象。

华清远见 HQYJ.COM
华清远见|宇宙实验中心 yyzlab.com.cn

3. **清空**按钮：用来将传送带上的物体全部清空。

4. **运行**按钮：在MQTT服务连接好的前提下，点击该按钮，会让上货杆开始上货。

华清远见宇宙实验中心 yyzlab.com.cn

5. **传送带速度**：可以直接在输入框输入固定的速度，也可以在右侧的滑动条进行滑动调节传送带的速度，其范围是0.3-3。

6. 在右下角可以去修改上货的货物的种类及每一个种类所对应的不同的类型。

本场景是基于桃子分类所做的模拟场景，其中桃子的类别共有**三类**，分别是**生的**、**半生不熟的**和**熟的**。其中每一个类别都有5种不同的桃子，且每次推出来的桃子的角度也不一定相同。

华清远见 HQYJ.COM
华清远见云实验中心 yyzlab.com.cn

在场景启动后，当有货物通过摄像头时，3D场景将右侧**采集区域图像**里面的图像编码成base64格式的数据，然后组装为JSON协议发送到MQTT服务中，其中，图像类的数据的键永远都是**image**，比如：{'image': 'xxxx'}，因此在使用代码接收时，可以去寻找**image**的存在。

因此，我们就可以通过Python程序去接受3D场景传上来的图片数据，并通过右侧有意的修改其上货的种类及类型，从而采集到所有的数据，以此来制作后续训练神经网络所需要的数据集。

1.3.2 分拣模式

点击**分拣模式**按钮，进入**分拣模式**。

进入场景后，首先通过**W、A、S、D键**和**鼠标右键**调整视角，使其视角如下图所示：

该场景中的**模式选择**、**设置**、**清空**、**启动**按钮及**传送带速度**与采集模式是相同的，只不过点击**启动**按钮后，系统不会自己推出货物，而是需要通过我们自己的代码去控制，并且上货的种类和类型也是随机的。并且我们的代码还需要包括**传送带的启动与停止**、**上货杆上货**、**分类杆分类**等内容，通信方式使用MQTT服务进行通信。

除此之外，多了一个**开始考试**的按钮，点击该按钮，在点击**确认**可以去进行一个模拟考试，方便我们对于我们的算法有一个准确的评估，它会计算规定时间内上货杆一共上了多少个货物以及分类的准确率，其中时间可以通过下拉框去进行选择。

倒计时和**结果**位于右上角。

当倒计时结束后，会自动弹出一个结果，包括规定时间内**上货的个数**、**分拣成功的数量**、**分拣成功的百分比**，以此来反映我们的逻辑控制程序及所训练的网络的分类能力。

华清远见官方实验中心 yyzlab.com.cn

1.3.3 考试模式

考试模式一般情况下是不开放的，只有安排考试才会进行开放，该模式下的考核内容和分拣模式中的**开始考试**所考核的内容是相同，不同的是，**分拣模式**下的**开始考试**的考核结果不会上传，而考试模式下的考核结果会上传到教务系统中进行系统性的记录。

2. MQTT通信协议介绍

2.1 MQTT通信协议的原理

2.1.1 MQTT简介

MQTT（Message Queuing Telemetry Transport）是一种轻量级的、基于发布/订阅模式的消息传输协议，专为低带宽、不可靠网络环境下的设备通信设计，尤其适用于物联网（IoT）设备之间的通信，基于TCP/IP的。

2.1.2 MQTT架构

MQTT的架构主要包括三个部分：客户端（Client）、代理（Broker）和主题（Topic）。

- **客户端（Client）**：任何连接到MQTT代理的设备都被称为客户端，客户端可以是发布者（Publisher）或订阅者（Subscriber）。
- **代理（Broker）**：代理是MQTT协议的核心，它负责接收、过滤和分发消息给订阅者。代理确保消息从发布者传递到一个或多个订阅者。

华清远见 HQYJ.COM
华清远见嵌入式学院实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- **主题（Topic）**：主题是消息的类别或地址。发布者将消息发布到特定的主题，而订阅者订阅他们感兴趣的主题以接收相关消息。

2.1.3 发布/订阅模式

- **发布者（Publisher）**：发布者向代理发布消息，并指定消息的主题。
- **订阅者（Subscriber）**：订阅者向代理订阅一个或多个主题，以接收相应的消息。
- **代理（Broker）**：代理接收发布者的消息，并将其分发给订阅了相应主题的订阅者。

2.1.4 MQTT消息格式

MQTT消息由以下几个部分组成：

- **固定头部（Fixed Header）**：包括消息类型、标志和剩余长度。
- **可变头部（Variable Header）**：根据不同类型的消息包含不同的信息，例如主题名、消息标识符等。
- **有效载荷（Payload）**：实际传递的数据内容。

2.1.5 QoS（服务质量）级别

MQTT支持三种服务质量（QoS）级别：

- **QoS 0**：最多一次（At most once）。消息不进行确认，可能会丢失。
- **QoS 1**：至少一次（At least once）。消息至少传递一次，但可能会重复。
- **QoS 2**：只有一次（Exactly once）。消息保证只传递一次，确保不重复。

2.1.6 保持连接（Keep Alive）

MQTT客户端和代理之间通过心跳包来保持连接，确保双方的连接状态。如果在设定的时间内没有心跳包传递，连接将被认为是断开。

2.2 如何使用MQTT

2.2.1 环境准备

- **MQTT代理**：你可以选择自托管的代理（如Eclipse Mosquitto）或使用云服务提供的代理（如HiveMQ、AWS IoT等），在这里大家打开本地服务管理平台，点击启动即可开启代理。

华清远见嵌入式实验中心 yyzlab.com.cn

- **客户端库**：根据你的编程语言选择合适的MQTT客户端库。例如，Python可以使用`paho-mqtt`库。

2.2.2 使用Python进行MQTT通信

- 安装`paho-mqtt`库：

```
python -m pip install paho-mqtt
```

- 发布消息的示例代码：

```
import paho.mqtt.client as mqtt

broker = "127.0.0.1"
port = 21883
topic = "test/topic"

# 发布的消息内容
message = 'Hello, MQTT!'

def on_connect(client, userdata, flags, rc):
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    if rc == 0:
        print("Connected successfully.")
    else:
        print("Connection failed with code %d." % rc)

# 创建MQTT客户端实例
client = mqtt.Client()

# 设置连接回调函数
client.on_connect = on_connect

# 连接到MQTT代理
client.connect(broker, port, 60)

# 启动网络循环
client.loop_start()

# 发布消息
client.publish(topic, message)

client.loop_stop()
```

- 订阅消息的示例代码：

```
import paho.mqtt.client as mqtt

broker = "127.0.0.1"
port = 21883
topic = "test/topic"

# 当连接到MQTT代理时调用
def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("Connected successfully.")
        # 订阅主题
        client.subscribe(topic)
    else:
        print("Connection failed with code %d." % rc)

# 当接收到消息时调用
def on_message(client, userdata, msg):
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    print(f"Received message: {msg.payload.decode()} on topic
{msg.topic}")

# 创建MQTT客户端实例
client = mqtt.Client()

# 设置连接和消息回调函数
client.on_connect = on_connect
client.on_message = on_message

# 连接到MQTT代理
client.connect(broker, port)

# 启动网络循环
client.loop_forever()
```

2.3 常见的应用场景

- **物联网（IoT）设备数据传输**：MQTT常用于传感器数据的实时传输和监控。
- **实时消息通知**：如推送通知、即时聊天应用等。
- **远程控制**：远程控制设备，如智能家居中的灯光、温度调节等。

3. 3D场景中的MQTT协议

3.1 控制类协议

指用户通过程序下发给3D场景控制推杆或传送带等控制类协议，topic为：**sim_device_01/device_control**

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

功能	JSON
推出一号推杆	{"rod_control":"first_push"}
拉回一号推杆	{"rod_control":"first_pull"}
推出二号推杆	{"rod_control":"second_push"}
拉回二号推杆	{"rod_control":"second_pull"}
推出三号推杆	{"rod_control":"third_push"}
拉回三号推杆	{"rod_control":"third_pull"}
推出四号推杆	{"rod_control":"fourth_push"}
拉回四号推杆	{"rod_control":"fourth_pull"}
推出所有推杆	{"rod_control":"all_push"}
拉回所有推杆	{"rod_control":"all_pull"}
传送带运行	{"conveyor":"run"}
传送带停止	{"conveyor":"stop"}

3.2 反馈类协议

指3D场景的光电门信息的上传协议，topic为：**sim_device_01/device_feedback**

功能	JSON
一号光电开关有遮挡	{”first_switch”:True}
一号光电开关无遮挡	{”first_switch”:False}
二号光电开关有遮挡	{”second_switch”:True}
二号光电开关无遮挡	{”second_switch”:False}
三号光电开关有遮挡	{”third_switch”:True}
三号光电开关无遮挡	{”third_switch”:False}

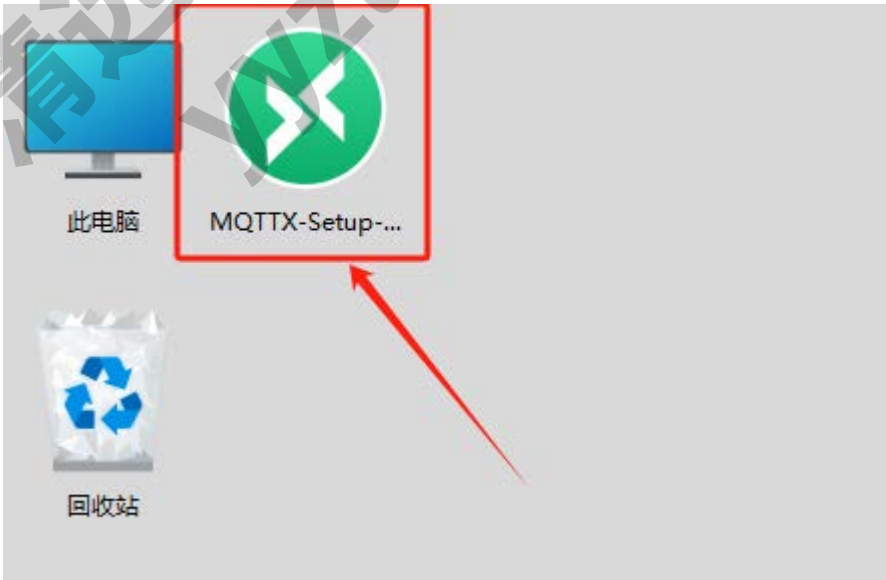
4. 3D场景通信控制与数据回传

华清远见元宇宙实验中心 yyzlab.com.cn

4.1 安装mqttx

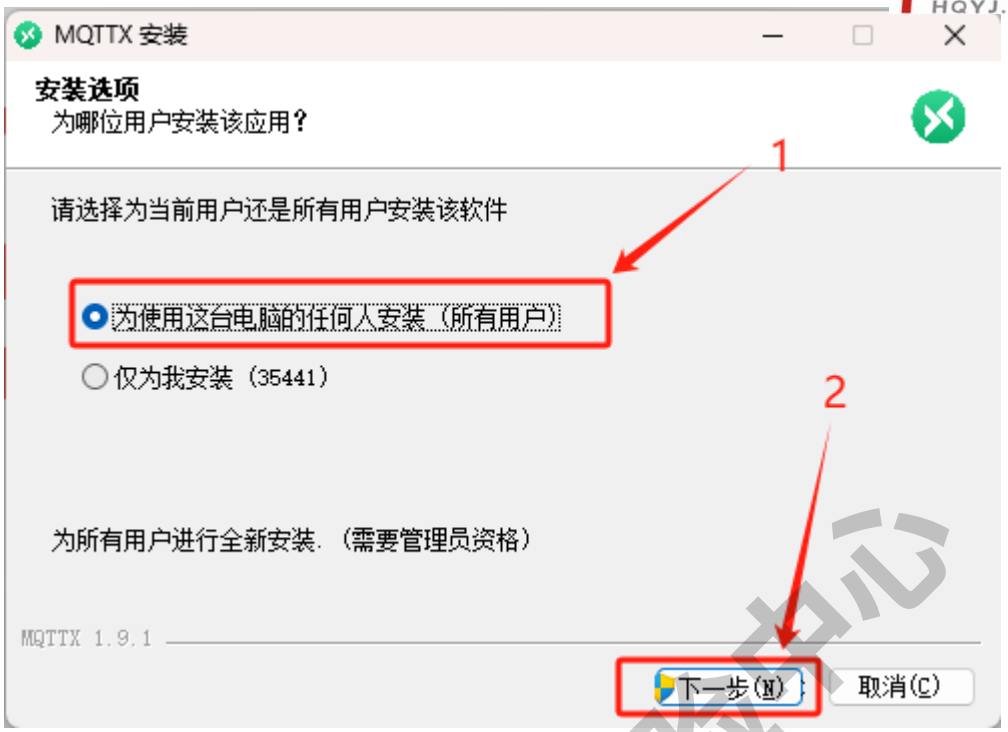
该软件安装包在**微信群公告中的技术支持相关**的群文件中。

下载好之后，双击exe文件进行安装。

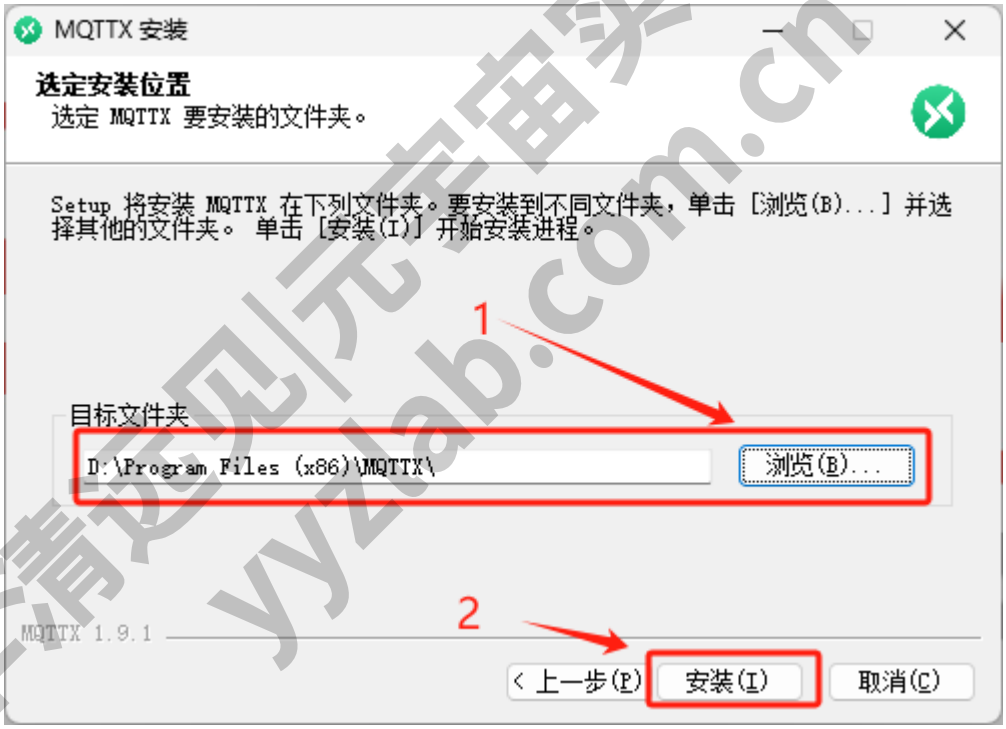


1. 进入安装程序，选择**为使用这台电脑的任何人安装**，然后点击**下一步**。

华清远见|元宇宙实验中心 yyzlab.com.cn

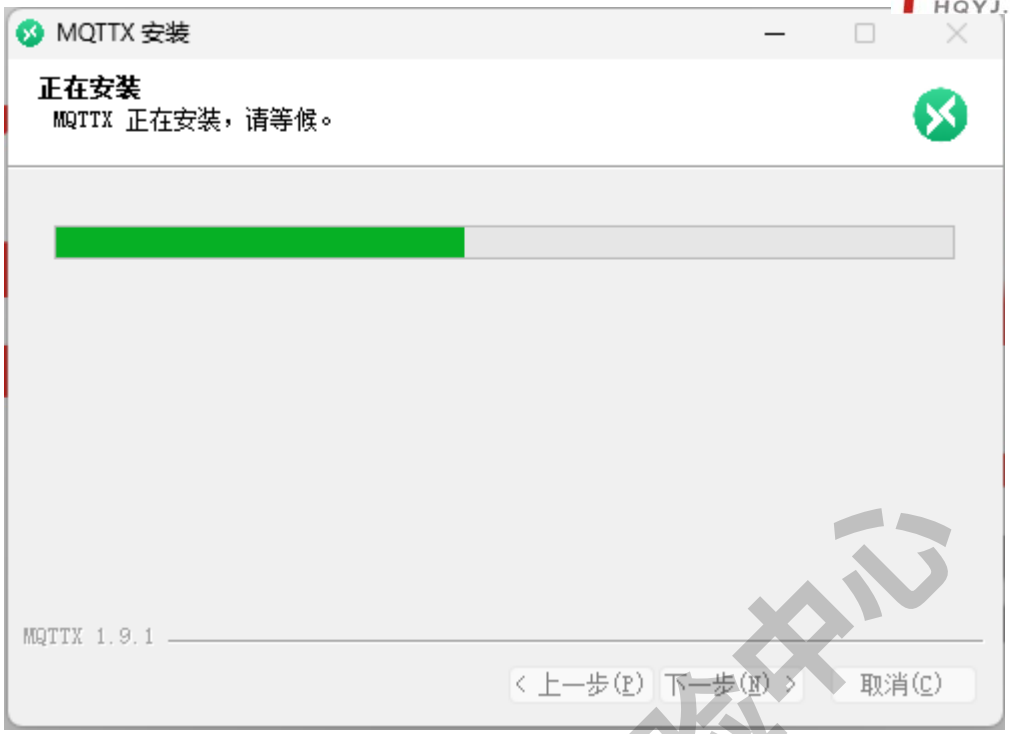


2. 点击**浏览**按钮，选择要安装的路径，然后点击**安装**按钮。



3. 等待安装完毕，点击**完成**按钮即可。

华清远见元宇宙实验中心 yyzlab.com.cn



华清远见元宇宙实验中心
yyzlab.com.cn

4.2 使用Python实现3D场景通信控制与数据回传

4.2.1 启动服务

打开**华清远见人工智能虚拟仿真本地服务管理平台**，登录后点击**启动**按钮。

华清远见元宇宙实验中心
yyzlab.com.cn

4.2.2 进入3D场景

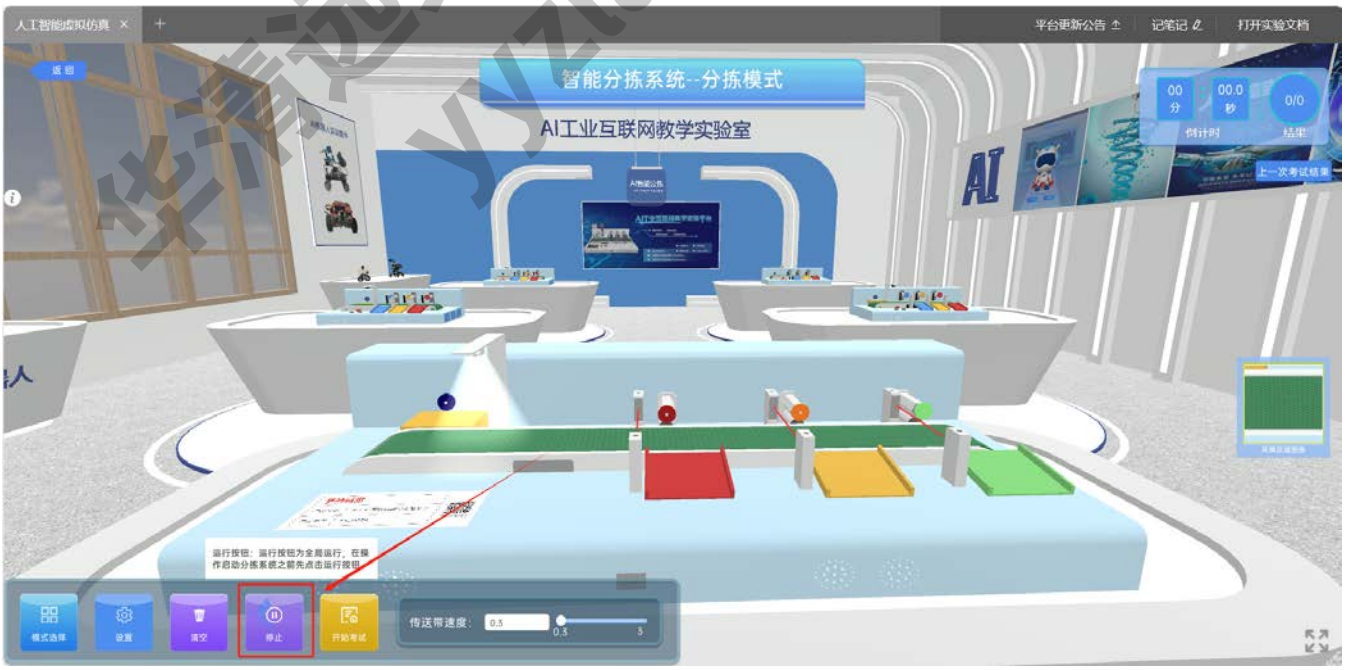
进入3D场景，并选择**分拣模式**，然后点击左下角的**设置**按钮。

在弹出的设置面板中，将**ip地址**改为 www.ws2wss.com或者**127.0.0.1**，并把**Port**改成29002，在确保上一步服务启动成功的前提下，点击**连接按钮**。

华清远见厦门实验中心 yvzlab.com.cn

此时，**连接**会变成**断开**，代表连接MQTT服务连接成功，然后将下面的**SubTopic**和**PubTopic**改成比较简单话题，方便我们区分，然后点击订阅，订阅完成后，如下图所示。

MQTT订阅成功后，就可以关闭设置面板了，然后点击左下角的**启动**按钮，就可以去与Python代码互动了。



华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

4.2.3 控制传送带运行与停止

想要控制传送带运行和停止，那么根据上面的协议就是：

传送带运行	{"conveyor":"run"}
传送带停止	{"conveyor":"stop"}

于是可以运行下面的代码去控制3D场景中的传送带运行并在5秒后停止。

```
import json
import paho.mqtt.client as mqtt
import time


class Mqtt_Clt():
    """
    用于表征Mqtt客户端的类
    """

    def __init__(self, ip_broker, port_broker, topic_sub,
topic_pub, time_out_secs):
        """
        parameters:ip_broker:broker的IP地址
                port_broker:连接服务的端口
                topic_sub:订阅话题
                topic_pub:发布话题
                time_out_secs:连接超时的时间
        """
        self.mqtt_clt = mqtt.Client()  # 构造Mqtt客户端
        # self.mqtt_clt.on_connect = self.on_connect
        # self.mqtt_clt.on_connect_fail = self.on_connect_fail
        self.mqtt_clt.on_message = self.on_message  # 设置Mqtt客户端
回调on_message为本类成员方法on_message
        self.topic_sub = topic_sub  # 设置订阅话题
        self.topic_pub = topic_pub  # 设置发布话题
        self.msg = {}  # 初始化接收消息的字典类成员变量msg为空({})
        self.mqtt_clt.connect(ip_broker, port_broker,
time_out_secs)  # 连接Mqtt Broker
        self.mqtt_clt.subscribe(self.topic_sub, qos=0)  # 订阅相关话
题
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
        self.mqtt_clt.loop_start()  # 开启接收循环

    def __del__(self):
        self.mqtt_clt.loop_forever()  # 结束接收循环
        self.mqtt_clt.disconnect()  # 断开连接

    def on_message(self, client, userdata, message):
        """
        parameters:client:回调返回的客户端实例
                   userdata:Client()或user_data_set()中设置的私有用户
数据
                   msg:MQTTMessage实例,包含topic,payload,qos,retain
        functions:接收消息的回调,提取消息中的相关内容
        """
        pass
        #
self.mqtt_queue.put(json.loads(message.payload.decode()))

    def send_json_msg(self, msg):
        """
        parameters:msg:json消息
        returns:无
        functions:在指定的话题上发布消息
        """
        self.mqtt_clt.publish(self.topic_pub, payload="
{}".format(msg))

    def control_device(self, str_key, str_value):
        """
        parameters:str_key:键的字符串
                   str_value:值的字符串
        functions:控制设备
        """
        self.send_json_msg(json.dumps({str_key: str_value}))


if __name__ == '__main__':
    mqtt_client = Mqtt_Clt("127.0.0.1", 21883, "bb", "aa", 60)
    # message_run = {"conveyor":"run"}
    mqtt_client.control_device("conveyor", 'run')
    time.sleep(5)
    # message_stop = {"conveyor":"stop"}
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    mqtt_client.control_device("conveyor", 'stop')
```

4.2.4 控制上货杆上货

上货杆就是1号杆，而上货前需要将传送带运行起来，所以根据上面的协议可得

传送带运行	{"conveyor":"run"}
推出一号推杆	{"rod_control":"first_push"}
拉回一号推杆	{"rod_control":"first_pull"}

那么运行下面的代码即可实现上货的功能。

```
import json
import paho.mqtt.client as mqtt
import time


class Mqtt_Clt():
    """
    用于表征Mqtt客户端的类
    """

    def __init__(self, ip_broker, port_broker, topic_sub,
topic_pub, time_out_secs):
        """
        parameters:ip_broker:broker的IP地址
                   port_broker:连接服务的端口
                   topic_sub:订阅话题
                   topic_pub:发布话题
                   time_out_secs:连接超时的时间
        """
        self.mqtt_clt = mqtt.Client()  # 构造Mqtt客户端
        # self.mqtt_clt.on_connect = self.on_connect
        # self.mqtt_clt.on_connect_fail = self.on_connect_fail
        self.mqtt_clt.on_message = self.on_message  # 设置Mqtt客户端
回调on_message为本类成员方法on_message
        self.topic_sub = topic_sub  # 设置订阅话题
        self.topic_pub = topic_pub  # 设置发布话题
        self.msg = {}  # 初始化接收消息的字典类成员变量msg为空({})
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
        self.mqtt_clt.connect(ip_broker, port_broker,
time_out_secs)  # 连接Mqtt Broker
        self.mqtt_clt.subscribe(self.topic_sub, qos=0)  # 订阅相关话
题
        self.mqtt_clt.loop_start()  # 开启接收循环

    def __del__(self):
        self.mqtt_clt.loop_forever()  # 结束接收循环
        self.mqtt_clt.disconnect()  # 断开连接

    def on_message(self, client, userdata, message):
        """
        parameters:client:回调返回的客户端实例
                   userdata:Client()或user_data_set()中设置的私有用户
数据
                   msg:MQTTMessage实例,包含topic,payload,qos,retain
        functions:接收消息的回调,提取消息中的相关内容
        """
        pass
        #
self.mqtt_queue.put(json.loads(message.payload.decode()))

    def send_json_msg(self, msg):
        """
        parameters:msg:json消息
        returns:无
        functions:在指定的话题上发布消息
        """
        self.mqtt_clt.publish(self.topic_pub, payload="
{}".format(msg))

    def control_device(self, str_key, str_value):
        """
        parameters:str_key:键的字符串
                   str_value:值的字符串
        functions:控制设备
        """
        self.send_json_msg(json.dumps({str_key: str_value}))


if __name__ == '__main__':
    mqtt_client = Mqtt_Clt("127.0.0.1", 21883, "bb", "aa", 60)
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    # 传送带运行  {"conveyor":"run"}
    # 推出1号杆  {"rod_control":"first_push"}
    # 拉回1号杆  {"rod_control":"first_pull"}
    mqtt_client.control_device("conveyor", 'run')
    while True:
        mqtt_client.control_device("rod_control", "first_push")
        time.sleep(1)
        mqtt_client.control_device("rod_control", "first_pull")
        time.sleep(1)
```

4.2.5 控制分类杆分类

分类杆就是后面的2、3、4号杆，所以根据上面的协议可得：

推出二号推杆	**{"rod_control":"second_push"}**
拉回二号推杆	{"rod_control":"second_pull"}
推出三号推杆	{"rod_control":"third_push"}
拉回三号推杆	{"rod_control":"third_pull"}
推出四号推杆	{"rod_control":"fourth_push"}
拉回四号推杆	{"rod_control":"fourth_pull"}

那么运行下面的代码即可实现分类杆的控制

```
import json
import paho.mqtt.client as mqtt
import time


class Mqtt_Clt():
    """
    用于表征Mqtt客户端的类
    """

    def __init__(self, ip_broker, port_broker, topic_sub,
topic_pub, time_out_secs):
        """
        parameters:ip_broker:broker的IP地址
                   port_broker:连接服务的端口
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
                    topic_sub:订阅话题
                    topic_pub:发布话题
                    time_out_secs:连接超时的时间
        """
        self.mqtt_clt = mqtt.Client()  # 构造Mqtt客户端
        # self.mqtt_clt.on_connect = self.on_connect
        # self.mqtt_clt.on_connect_fail = self.on_connect_fail
        self.mqtt_clt.on_message = self.on_message  # 设置Mqtt客户端
回调on_message为本类成员方法on_message
        self.topic_sub = topic_sub  # 设置订阅话题
        self.topic_pub = topic_pub  # 设置发布话题
        self.msg = {}  # 初始化接收消息的字典类成员变量msg为空({})
        self.mqtt_clt.connect(ip_broker, port_broker,
time_out_secs)  # 连接Mqtt Broker
        self.mqtt_clt.subscribe(self.topic_sub, qos=0)  # 订阅相关话
题
        self.mqtt_clt.loop_start()  # 开启接收循环

    def __del__(self):
        self.mqtt_clt.loop_forever()  # 结束接收循环
        self.mqtt_clt.disconnect()  # 断开连接

    def on_message(self, client, userdata, message):
        """
        parameters:client:回调返回的客户端实例
                    userdata:Client()或user_data_set()中设置的私有用户
数据
                    msg:MQTTMessage实例,包含topic,payload,qos,retain
        functions:接收消息的回调,提取消息中的相关内容
        """
        pass
        #
self.mqtt_queue.put(json.loads(message.payload.decode()))

    def send_json_msg(self, msg):
        """
        parameters:msg:json消息
        returns:无
        functions:在指定的话题上发布消息
        """
        self.mqtt_clt.publish(self.topic_pub, payload="
{}".format(msg))
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    def control_device(self, str_key, str_value):
        """
        parameters:str_key:键的字符串
                   str_value:值的字符串
        functions:控制设备
        """
        self.send_json_msg(json.dumps({str_key: str_value}))


if __name__ == '__main__':
    mqtt_client = Mqtt_Clt("127.0.0.1", 21883, "bb", "aa", 60)
    # 推出二号推杆 {"rod_control": "second_push"}
    # 拉回二号推杆 {"rod_control": "second_pull"}
    # 推出三号推杆 {"rod_control": "third_push"}
    # 拉回三号推杆 {"rod_control": "third_pull"}
    # 推出四号推杆 {"rod_control": "fourth_push"}
    # 拉回四号推杆 {"rod_control": "fourth_pull"}
    mqtt_client.control_device("conveyor", 'run')
    while True:
        mqtt_client.control_device("rod_control", "second_push")
        time.sleep(1)
        mqtt_client.control_device("rod_control", "second_pull")
        time.sleep(1)
        mqtt_client.control_device("rod_control", "third_push")
        time.sleep(1)
        mqtt_client.control_device("rod_control", "third_pull")
        time.sleep(1)
        mqtt_client.control_device("rod_control", "fourth_push")
        time.sleep(1)
        mqtt_client.control_device("rod_control", "fourth_pull")
        time.sleep(1)
```

4.2.6 接收3D场景中光电对管消息和图片信息

通过传回来的消息结合反馈类协议去判断当前货物所处的位置，如下面代码所示：

```
import json
import paho.mqtt.client as mqtt
import time
import queue
import threading
```

华清远见元宇宙实验中心 yyzlab.com.cn

```
class Mqtt_Clt():
    """
    用于表征Mqtt客户端的类
    """

    def __init__(self, ip_broker, port_broker, topic_sub,
topic_pub, time_out_secs):
        """
        parameters:ip_broker:broker的IP地址
                   port_broker:连接服务的端口
                   topic_sub:订阅话题
                   topic_pub:发布话题
                   time_out_secs:连接超时的时间
        """
        self.mqtt_clt = mqtt.Client()  # 构造Mqtt客户端
        # self.mqtt_clt.on_connect = self.on_connect
        # self.mqtt_clt.on_connect_fail = self.on_connect_fail
        self.mqtt_clt.on_message = self.on_message  # 设置Mqtt客户端
回调on_message为本类成员方法on_message
        self.topic_sub = topic_sub  # 设置订阅话题
        self.topic_pub = topic_pub  # 设置发布话题
        self.msg = {}  # 初始化接收消息的字典类成员变量msg为空({})
        self.mqtt_queue = queue.Queue(5)
        self.mqtt_clt.connect(ip_broker, port_broker,
time_out_secs)  # 连接Mqtt Broker
        self.mqtt_clt.subscribe(self.topic_sub, qos=0)  # 订阅相关话
题
        self.mqtt_clt.loop_start()  # 开启接收循环

    def __del__(self):
        self.mqtt_clt.loop_forever()  # 结束接收循环
        self.mqtt_clt.disconnect()  # 断开连接

    def on_message(self, client, userdata, message):
        """
        parameters:client:回调返回的客户端实例
                   userdata:Client()或user_data_set()中设置的私有用户
数据
                   msg:MQTTMessage实例,包含topic,payload,qos,retain
        functions:接收消息的回调,提取消息中的相关内容
        """
```

华清远见嵌入式实验中心 yyzlab.com.cn

```
        self.mqtt_queue.put(json.loads(message.payload.decode()))

    def send_json_msg(self, msg):
        """
        parameters:msg:json消息
        returns:无
        functions:在指定的话题上发布消息
        """
        self.mqtt_clt.publish(self.topic_pub, payload="
{}".format(msg))

    def control_device(self, str_key, str_value):
        """
        parameters:str_key:键的字符串
                   str_value:值的字符串
        functions:控制设备
        """
        self.send_json_msg(json.dumps({str_key: str_value}))

def get_message(mqtt_queue):
    while True:
        msg = mqtt_queue.get()
        print(msg)


if __name__ == '__main__':
    mqtt_client = Mqtt_Clt("127.0.0.1", 21883, "bb", "aa", 60)
    mqtt_client.control_device("conveyor", 'run')

    t_get_msg = threading.Thread(target=get_message, args=
[mqtt_client.mqtt_queue])
    t_get_msg.start()


    while True:
        mqtt_client.control_device("rod_control", "first_push")
        time.sleep(1)
        mqtt_client.control_device("rod_control", "first_pull")
        time.sleep(1)
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

华清远见|元宇宙实验中心
yyzlab.com.cn